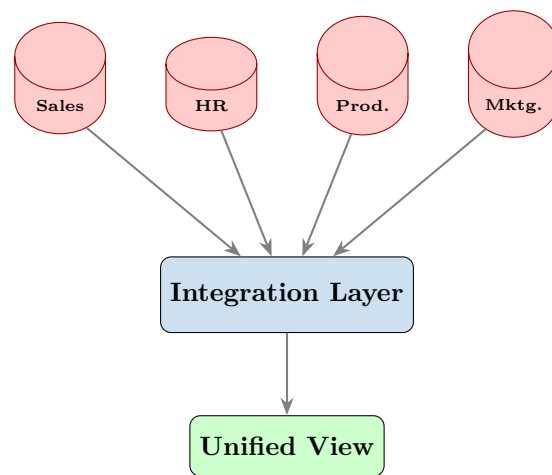


Data Integration

Complete Exam-Ready Study Notes

F20BD / F21BD – Big Data Management

Heriot-Watt University



Contents

1	The Problem: Data Silos	2
1.1	Why Do Data Silos Exist?	2
1.2	Problems Caused by Data Silos	2
2	Data Integration: The Solution	3
2.1	Six Approaches to Data Integration	3
3	Data Warehousing	4
3.1	ETL vs ELT	4
3.2	Star Schema vs Snowflake Schema	5
4	Virtual Data Integration (VDI)	6
4.1	VDI Architecture: Mediator-Wrapper	6
4.2	Characteristics of VDI	7
4.3	Data Warehousing vs Virtual Data Integration	7
5	Mapping Approaches: GAV, LAV, GLAV	8
6	Semantic Data Integration & OBDA	9
6.1	Ontology-Based Data Access (OBDA)	9
6.2	Database-to-Ontology Mapping Approaches	9
6.3	R2RML (RDB to RDF Mapping Language)	10
7	The Ontop Framework and Protégé (Lab)	11
7.1	What Is Ontop?	11
7.2	What Is Protégé?	11
7.3	The Lab Setup: Three Files You Need	11
7.4	The University Ontology (from university.ttl)	11
7.5	The Database (H2 – uni1 schema)	12
7.6	The Mappings (How DB Maps to Ontology)	12
7.7	SPARQL Query Example from the Lab	13
8	Quick Reference Cheat Sheet	14
8.1	Key Concepts at a Glance	14
8.2	Practice Questions	15

The Problem: Data Silos

The Department Filing Cabinets Imagine a company where every department (HR, Sales, Marketing, Production, Customer Service) has its own locked filing cabinet. HR stores employee records, Sales stores customer purchases, Marketing stores campaign data. Nobody can look into anyone else's cabinet. The CEO wants a company-wide report combining sales figures with marketing campaigns and HR data — but the cabinets are all separate, using different label systems, and nobody shares keys. **These isolated filing cabinets are data silos.**

What Are Data Silos?

Data silos are repositories of information that are useful to the business unit that creates them, but **inaccessible to other teams** that might benefit from the data they hold. In simple terms: different departments store their own data in their own systems, and these systems **don't talk to each other**.

Why Do Data Silos Exist?

Four Causes of Data Silos

Cause	Explanation
Legacy systems & technical debt	Old databases and systems that have been in use for years. Nobody wants to replace them because it is expensive and risky.
Organisational silos & politics	Departments don't share data due to office politics, leadership issues, or lack of data governance. "That's our data, not yours."
Inconsistent data standards	Different departments use different schemas, formats, and naming conventions. Sales calls it "Customer Name", HR calls it "Employee Name", Marketing calls it "Contact Name".
Lack of data strategy	No overarching plan for how data should be managed, shared, or governed across the whole organisation.

Problems Caused by Data Silos

Why Silos Are Dangerous

- **No overview:** Marketing cannot see Sales data to optimise campaigns.
- **Duplicated data:** Same customer stored in 5 different systems = wasted storage and effort. In hospitals, fixing duplicated patient records costs up to \$1M per year.
- **Inconsistent data:** Patient "John Smith" is "J. Smith" in one system and "Smith, John" in another. Which is correct? Inconsistency can cause real harm (e.g., wrong medical treatment).
- **Wasted resources:** Staff spend time manually copying data between systems.

Data Integration: The Solution

What Is Data Integration?

Data integration is the process of **combining data from several disparate (different) sources** to provide a **unified view** of the data. Once unified, the data can be used for analysis, reporting, and data mining.

Six Approaches to Data Integration

Know These Six Approaches

Approach	Description
Manual & Middleware	Humans or middleware software manually move and transform data between systems. Labour-intensive and error-prone.
Application-Based	Custom applications are built to connect specific systems. Works but hard to scale.
Data Federation	Query multiple data sources in real-time without moving data. Uses a virtual unified view.
Data Virtualisation	Similar to federation; creates a virtual layer that lets users query data as if it were in one place, without physically moving it.
Data Warehousing	Physically copy all data into one central database (the “warehouse”) on a regular schedule.
ETL / ELT	Extract-Transform-Load (or Extract-Load-Transform). The process used to move data into a warehouse.

Data Warehousing

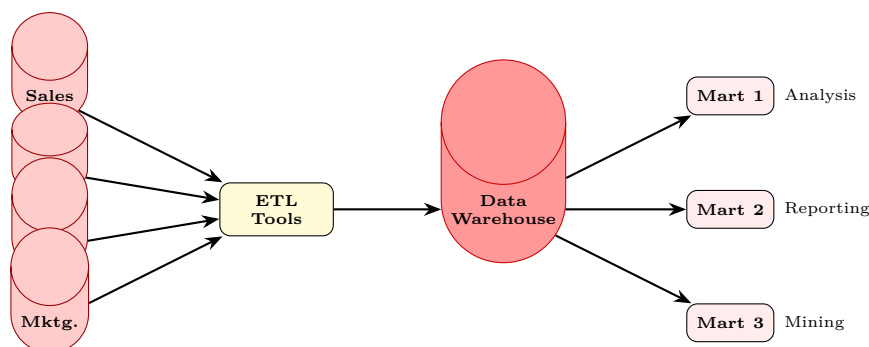
The Central Library Imagine every department photocopies its filing cabinet contents every night and sends the copies to a central library. Now anyone can walk into the central library and find data from *all* departments in one place. The downside? The copies might be slightly outdated (they were made last night), and you need a big library to hold everything. **This central library is a data warehouse.**

Data Warehouse Architecture

A **data warehouse** makes **periodical physical copies** of data from multiple sources into a single, integrated database.

Key components:

- **Data Sources** — The original departmental databases (Sales, HR, Production, etc.), each with its own **local schema**.
- **ETL Tools** — Software that Extracts data from sources, Transforms it into a common format, and Loads it into the warehouse.
- **Data Warehouse** — The central integrated database with a **global schema**.
- **Data Marts** — Smaller subsets of the warehouse focused on a specific topic (e.g., a Sales data mart for the Sales team).
- **Analysis / Reporting / Mining** — What the business actually does with the unified data.



ETL vs ELT

ETL: Extract, Transform, Load

ETL is the three-step process for moving data into a warehouse:

1. **Extract:** Read data from each source system (requires specific software connectors for each source).
2. **Transform:** Convert the extracted data into the target warehouse's format. This includes cleaning (fix errors), merging (combine duplicates), applying rules, and calculating new fields.
3. **Load:** Write the transformed data into the data warehouse.

ELT reverses the last two steps: Extract, then **Load** the raw data into a data lake first, **then Transform** it inside the lake/warehouse. ELT is popular with cloud-based systems that have massive processing power.

ETL vs ELT — Know the Difference

	ETL	ELT
Order	Extract → Transform → Load	Extract → Load → Transform
Where transform happens	In a separate ETL server/tool	Inside the target database
Best for	Traditional data warehouses	Cloud data lakes (e.g., BigQuery)
Example tools	Talend, Informatica, SSIS	BigQuery, Snowflake, Redshift

Star Schema vs Snowflake Schema

Data warehouses typically use **dimensional modelling** to organise data. There are two main schemas:

Star Schema

A **star schema** has a central **fact table** (containing measurable data like sales amounts) connected directly to **dimension tables** (containing descriptive data like product name, date, store location). The shape looks like a star.

Characteristics: Simple, fast queries (fewer JOINS), some redundant data, best for small data marts, easy to understand.

Snowflake Schema

A **snowflake schema** is an extension of the star schema where dimension tables are **further normalised** (split into sub-tables). For example, a “Time” dimension might be split into Year, Month, Week, and Weekday sub-tables. The shape resembles a snowflake.

Characteristics: More complex, slower queries (more JOINS), less redundant data (more storage-efficient), allows finer-grained analysis.

Star vs Snowflake Comparison

Feature	Star Schema	Snowflake Schema
Complexity	Simple	More complex
Query Performance	Faster (fewer JOINS)	Slower (more JOINS)
Storage Efficiency	Less efficient (redundant data)	More efficient (normalised)
Ease of Use	Easier to understand and query	Harder to understand and query
Best For	Data marts, reporting	Full warehouse, fine analysis

Virtual Data Integration (VDI)

The Translator at the UN Imagine a United Nations meeting where each country's representative speaks only their own language. Instead of making everyone learn every language, they hire a **translator** (mediator) who understands all languages. When someone asks a question, the translator reformulates it into each representative's language, gets answers, and combines them into a single response. **The data never leaves its home country — only the answers travel.**

Virtual Data Integration (VDI)

VDI allows users to access and query data from multiple different sources **without physically moving or replicating the data** into a centralised repository. The data stays at the source. A virtual “mediator” layer translates queries on-the-fly.

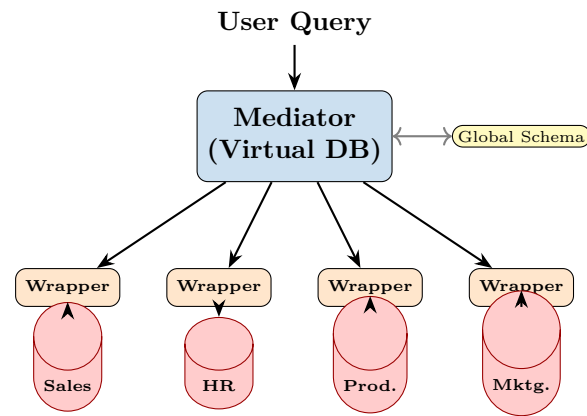
Key difference from data warehousing: In warehousing, data is physically copied to a central location. In VDI, data stays where it is — only queries and results move.

VDI Architecture: Mediator-Wrapper

The VDI architecture has three key components:

Mediator-Wrapper Architecture

Component	What It Does
Wrappers	A connector for each data source. It “wraps” the source so it can communicate with the mediator. It translates data from the source's local schema into the format the mediator expects.
Mediator	The central brain. It holds the global schema (the unified view). When a user sends a query, the mediator breaks it into sub-queries, sends them to the relevant wrappers, collects results, combines them, and returns a unified answer.
Global Schema	Also called the mediated schema . It describes the unified view of all the data as if it were in one database. Users write queries against this schema — they never need to know about the individual source schemas.
Source Mappings	Logical formulas (like SQL views) that describe how data in each local schema relates to the global schema. These mappings tell the mediator how to translate queries.



Characteristics of VDI

- Data sources are **updated independently** — nobody controls when a source changes its data.
- The same source can participate in **multiple mediated systems**.
- Sources can be **added or removed** without rebuilding the whole system.
- The mediated schema captures what data is available and how it maps to the global view.
- Data is **always up-to-date** (unlike warehousing, where it might be a day old).

Data Warehousing vs Virtual Data Integration

Key Comparison (Very Likely Exam Question)

Feature	Data Warehousing	Virtual Data Integration
Data location	Physically copied to central warehouse	Stays at the source (never moved)
Data freshness	May be stale (last ETL run)	Always up-to-date (live queries)
Query speed	Fast (pre-computed, local data)	Slower (must query remote sources)
Storage cost	High (duplicate all data)	Low (no duplication)
Setup complexity	High (build warehouse + ETL)	Moderate (write wrappers + mappings)
Adding sources	Requires new ETL pipelines	Add wrapper + mapping
Use case	Historical analysis, reporting	Real-time queries, integration

Mapping Approaches: GAV, LAV, GLAV

Mappings are the “instructions” that tell the mediator how to translate between local schemas and the global schema. There are three approaches:

Three Mapping Approaches

Approach	How It Works	When to Use
GAV (Global-As-View)	The global schema is defined as views over the local schemas . “The global table Employees is built by combining data from Source1.Staff and Source2.Personnel.”	When local sources are stable and well-known. Easy to answer queries, but hard to add new sources.
LAV (Local-As-View)	The local schemas are defined as views over the global schema . “Source1.Staff is a subset of the global Employees table.”	When sources change often or new sources are added frequently. Hard to answer queries, but easy to add sources.
GLAV (Global-and-Local-As-View)	Combines both: mappings are expressed as correspondences between views on both the global and local schemas.	Most flexible. GAV and LAV are special cases of GLAV.

GAV vs LAV **GAV** is like writing a recipe from ingredients you already have: “My unified salad = tomatoes from Garden A + lettuce from Garden B.” You define the *global* thing in terms of *local* things.

LAV is the opposite: “Garden A’s tomatoes are a type of vegetable in my universal cookbook.” You define each *local* thing in terms of the *global* thing. This makes it easy to add “Garden C” later — just say what it contributes to the cookbook.

Semantic Data Integration & OBDA

The Problem with Legacy Data

Most organisations have **huge amounts of data in relational databases (RDBMS)**. Importing all this into RDF triple stores would require enormous effort and cost. The question is: how can we make this relational data available on the Semantic Web **without physically converting it**? The answer: **expose relational databases as if they were RDF**, using ontologies and mappings.

Ontology-Based Data Access (OBDA)

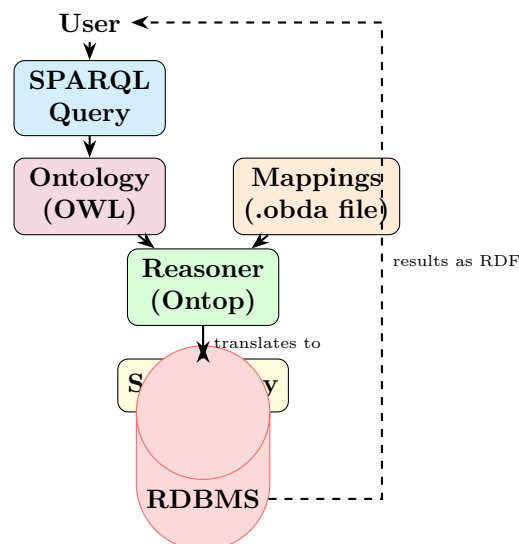
What Is OBDA?

Ontology-Based Data Access is a data integration approach where:

1. An **ontology** (written in OWL) defines a high-level, human-friendly global schema.
2. **Mappings** connect the ontology's vocabulary to the relational database tables/columns.
3. Users write queries using **SPARQL** (the Semantic Web query language) against the ontology.
4. A **reasoner** translates the SPARQL queries into **SQL**, runs them against the relational database, and returns results as RDF triples.

The user never needs to know SQL or the database structure — they only see the ontology!

The OBDA Analogy Imagine you speak only English, but your company's data is in a Chinese database. OBDA gives you an English “menu” (the ontology) with concepts you understand (“Student”, “Course”, “Professor”). When you ask a question in English, the system translates it to Chinese (SQL), gets the answer from the Chinese database, and gives you the result back in English (RDF). You never need to learn Chinese.



Database-to-Ontology Mapping Approaches

There are two main ways to connect a database to an ontology:

Approach	Description
Direct Mapping	Automatically generates RDF from a relational database using simple rules: each table becomes a class, each column becomes a property, each row becomes an instance. No custom ontology needed. Quick but inflexible.
R2RML Mapping	A W3C standard for custom mapping from relational databases to RDF. You write “triples maps” that specify exactly how each table/column maps to ontology classes/properties. More work, but full control.

R2RML (RDB to RDF Mapping Language)

What Is R2RML?

R2RML is a W3C standard language for expressing mappings from relational databases to RDF. It uses **triples maps** that define:

- **Logical table:** which database table (or SQL query) provides the source data.
- **Subject map:** how to construct the IRI (identifier) for each RDF subject from row data.
- **Predicate-Object map:** which ontology property to use and which column provides the value.

An R2RML file is processed by a **mapping engine** which extracts data from the database, transforms it using the mappings, and produces RDF triples.

Direct Mapping — Employees Table to RDF Given this table:

EmployeeID	Name	DepartmentID
1	John Doe	101
2	Jane Smith	102

Direct mapping automatically produces this RDF (Turtle syntax):

```
<http://example.com/Employees/1>
  a <http://example.com/Employees> ;
  <http://example.com/Employees#Name> "John Doe" ;
  <http://example.com/Employees#DepartmentID> "101" .

<http://example.com/Employees/2>
  a <http://example.com/Employees> ;
  <http://example.com/Employees#Name> "Jane Smith" ;
  <http://example.com/Employees#DepartmentID> "102" .
```

Each row becomes a subject (identified by the primary key), each column becomes a predicate, and each cell value becomes an object.

The Ontop Framework and Protégé (Lab)

What Is Ontop?

Ontop Framework

Ontop is an open-source OBDA system that creates **Virtual Knowledge Graphs (VKGs)**. It exposes relational database content as knowledge graphs **without physically moving the data**. When you send a SPARQL query, Ontop:

1. **Rewrites** the SPARQL query into SQL using the mappings and ontology.
2. **Delegates** the SQL query to the actual relational database.
3. **Returns** the results as RDF triples.

The graphs are **virtual**: data stays in the relational database. It is always up-to-date and never duplicated.

What Is Protégé?

Protégé is a free, open-source ontology editor used to create and manage OWL ontologies. In the lab, the Ontop plugin for Protégé lets you define mappings between your database and ontology, then run SPARQL queries.

The Lab Setup: Three Files You Need

The Three Key Files

File	Purpose
<code>university.ttl</code>	The ontology file (in Turtle format). Defines classes (Student, Course, Professor, etc.) and properties (teaches, attends, firstName, etc.).
<code>university.obda</code>	The mappings file. Defines how database tables map to ontology classes/properties using Target (Turtle template) and Source (SQL query) pairs.
<code>university.properties</code>	The connection file. Contains the JDBC URL, username, and password for the H2 database.

The University Ontology (from `university.ttl`)

The ontology defines this class hierarchy:

Class Hierarchy

```
owl:Thing
  +-- Course
  +-- EducationalInstitution
  +-- foaf:Person
      +-- FacultyMember
          +-- Researcher
              | +-- PostDoc
              | +-- Professor (also subclass of Teacher)
              | +-- AssistantProfessor
```

```

|           +-- AssociateProfessor
|           +-- FullProfessor
+-- Student
|   +-- GraduateStudent
|   |   +-- PhDStudent (also subclass of Researcher)
|   +-- UndergraduateStudent
+-- Teacher
    +-- ExternalTeacher
    +-- Professor (also listed above)

```

Object Properties: attends, isGivenAt, isSupervisedBy, isTaughtBy, teaches (with sub-properties givesLab and givesLecture).

Data Properties: foaf:firstName, foaf:lastName, title.

The Database (H2 – uni1 schema)

The H2 database contains these tables in the uni1 schema:

Table	What It Contains
uni1.student	Students (s_id, first_name, last_name)
uni1.academic	Academics (a_id, first_name, last_name, position) where position 1=FullProf, 2=AssocProf, etc.
uni1.course	Courses (c_id, title)
uni1.teaching	Which academic teaches which course (a_id, c_id)
uni1.course-registration	Which student attends which course (s_id, c_id)

The Mappings (How DB Maps to Ontology)

Each mapping has a **Target** (what RDF triples to create, using Turtle-like syntax with placeholders) and a **Source** (which SQL query to run to get the data).

All Six Mappings from the Lab

Mapping 1: uni1-student — Maps students to the Student class

```

Target: :uni1/student/{s_id} a :Student ;
        foaf:firstName {first_name}^^xsd:string ;
        foaf:lastName {last_name}^^xsd:string .
Source: SELECT * FROM "uni1"."student"

```

Explanation: For each row in the student table, create an RDF resource with the URI pattern :uni1/student/1 (using the student ID), type it as a :Student, and attach their first and last names.

Mapping 2: uni1-academic — Maps academics to FacultyMember class

```

Target: :uni1/academic/{a_id} a :FacultyMember ;
        foaf:firstName {first_name}^^xsd:string ;
        foaf:lastName {last_name}^^xsd:string .
Source: SELECT * FROM "uni1"."academic"

```

Mapping 3: uni1-course — Maps courses to Course class

```

Target: :uni1/course/{c_id} a :Course ;
        :title {title} ;
        :isGivenAt :uni1/university .
Source: SELECT * FROM "uni1"."course"

```

Mapping 4: uni1-teaching — Maps the teaches relationship

Target: :uni1/academic/{a_id} :teaches :uni1/course/{c_id} .
 Source: SELECT * FROM "uni1"."teaching"

Mapping 5: uni1-registration — Maps the attends relationship

Target: :uni1/student/{s_id} :attends :uni1/course/{c_id} .
 Source: SELECT * FROM "uni1"."course-registration"

Mapping 6: uni1-fullProfessor — Maps position=1 to FullProfessor

Target: :uni1/academic/{a_id} a :FullProfessor .
 Source: SELECT * FROM "uni1"."academic" WHERE "position" = 1

Explanation: Only academics with position=1 are classified as FullProfessor. The WHERE clause filters the source data.

How Mappings Work — Step by Step Let's trace what happens when we query for full professors:

1. User writes SPARQL: SELECT ?prof WHERE { ?prof a :FullProfessor }
2. Ontop looks at the mappings and finds **uni1-fullProfessor**: the source is SELECT * FROM "uni1"."academic" WHERE "position" = 1
3. Ontop rewrites the SPARQL into this SQL and runs it against the H2 database
4. The database returns rows (e.g., a_id=1, a_id=12)
5. Ontop uses the target template to construct RDF: :uni1/academic/1 a :FullProfessor and :uni1/academic/12 a :FullProfessor
6. Results are returned to the user

SPARQL Query Example from the Lab

“Give me the last names of full professors”

```
PREFIX : <http://example.org/voc#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?prof ?lastName {
  ?prof a :FullProfessor ;
        foaf:lastName ?lastName .
}
```

Behind the scenes: Ontop translates this into SQL:

```
SELECT "last_name", "first_name"
FROM "uni1"."academic"
WHERE "position" = 1;
```

Result: Returns “Chambers” and “Josephina” — the same result you would get from running SQL directly on the H2 database!

Quick Reference Cheat Sheet

Key Concepts at a Glance

Concept	One-Line Summary
Data Silo	Isolated data repository accessible only to its owning department.
Data Integration	Combining data from multiple sources into a unified view.
ETL	Extract-Transform-Load: the process of moving data into a warehouse.
ELT	Extract-Load-Transform: load raw data first, then transform inside the target system.
Data Warehouse	Central physical copy of all source data, updated periodically via ETL.
Data Mart	A subset of a warehouse focused on one department/topic.
Star Schema	Fact table at centre, dimension tables around it. Simple, fast, some redundancy.
Snowflake Schema	Like star but dimensions are normalised into sub-tables. Complex, less redundancy.
VDI	Virtual Data Integration: query sources without moving data. Uses mediator + wrappers.
Mediator	Central component that holds global schema and translates/routes queries.
Wrapper	Connector that translates a source's local schema for the mediator.
Global Schema	Unified schema that users query against (hides source complexity).
GAV	Global-As-View: global schema defined as views over local schemas.
LAV	Local-As-View: local schemas defined as views over global schema. Easy to add sources.
GLAV	Global-and-Local-As-View: combines both GAV and LAV. Most flexible.
OBDA	Ontology-Based Data Access: use ontology + mappings to query relational data via SPARQL.
Direct Mapping	Auto-generate RDF from database structure (table=class, column=property).
R2RML	W3C standard for custom RDB-to-RDF mappings using triples maps.
Ontop	Open-source OBDA system: translates SPARQL to SQL, returns RDF. Data stays in DB.
Protégé	Open-source ontology editor. Used with Ontop plugin for OBDA.
Virtual Knowledge Graph	Knowledge graph where data is not materialised; queries go to source DB live.

Practice Questions

Q1: Data Silos What is a data silo, and name two problems it causes?

Answer: A data silo is an isolated repository of data that is accessible only to the team that created it. Problems include: (1) no unified overview of company data, preventing cross-department insights; (2) duplicated and inconsistent data across systems, wasting storage and potentially causing errors.

Q2: Warehousing vs VDI Compare data warehousing and virtual data integration. Give one advantage of each.

Answer: Data warehousing physically copies all data to a central database, while VDI keeps data at the sources and queries them virtually. Advantage of warehousing: fast queries (data is local and pre-processed). Advantage of VDI: data is always up-to-date (queried in real-time from sources).

Q3: ETL Steps Describe the three steps of ETL.

Answer: (1) **Extract:** read data from each source system using connectors. (2) **Transform:** clean, merge, and convert the data into the target warehouse's format. (3) **Load:** write the transformed data into the data warehouse.

Q4: GAV vs LAV What is the difference between GAV and LAV mapping approaches?

Answer: In **GAV** (Global-As-View), the global schema is defined as views over the local schemas — you describe each global relation in terms of source data. In **LAV** (Local-As-View), each local source is defined as a view over the global schema. GAV makes query answering easier; LAV makes adding new sources easier.

Q5: OBDA What is OBDA and how does it work?

Answer: OBDA (Ontology-Based Data Access) uses an ontology as a high-level global schema over relational databases. Users write SPARQL queries against the ontology. A reasoner (like Ontop) uses mappings to translate SPARQL into SQL, runs the SQL on the database, and returns results as RDF triples. The data never leaves the source database.

Q6: Ontop Process Describe what happens step-by-step when a user runs a SPARQL query in Ontop.

Answer: (1) The user writes a SPARQL query against the ontology. (2) Ontop consults the mappings to find which database tables/columns correspond to the ontology terms. (3) Ontop rewrites the SPARQL into an SQL query. (4) The SQL is executed against the relational database (e.g., H2). (5) The database returns rows. (6) Ontop converts the rows into RDF triples using the mapping templates and returns them to the user.

Q7: Star vs Snowflake When would you use a star schema over a snowflake schema?

Answer: Use a star schema when you need simple, fast queries with fewer JOINS, typically for data marts and straightforward reporting with small to medium datasets. Use a snowflake schema

when storage efficiency matters and you need finer-grained dimensional analysis, accepting slower query performance due to more JOINS.

Q8: Mappings in OBDA In the university lab, the mapping for FullProfessor uses the SQL WHERE "position" = 1. Why is this WHERE clause needed?

Answer: The “academic” table stores all academics (full professors, associate professors, assistants, etc.) with a “position” column that encodes their rank (1=FullProfessor). The WHERE clause filters the rows so that only academics with position=1 are classified as FullProfessor in the ontology. Without it, all academics would be typed as FullProfessor.